

モナコノートコード集

特に意味もなく毎回作ってるよ！

コード

処理系

`rust

main.rs

```
#![cfg_attr(not(debug_assertions), windows_subsystem = "windows")]
```

```
fn main() { monaco_note_lib::run(); }
```

,

`rust

lib.rs

```
mod file_ops; mod font_ops;
```

```
#[cfg_attr(mobile, tauri::mobile_entry_point)] pub fn run() { tauri::Builder::default()
.plugin(tauri_plugin_shell::init()) .plugin(tauri_plugin_dialog::init()) .plugin(tauri_plugin_opener::init())
.invoke_handler(tauri::generate_handler![ file_ops::read_text_file, file_ops::write_text_file, font_ops::get_os_fonts,
font_ops::refresh_os_fonts, font_ops::load_config, font_ops::save_config, font_ops::get_config_path,
font_ops::save_tab_cache, font_ops::load_tab_cache ]) .run(tauri::generate_context!()) .expect("error while
running tauri application"); }
```

,

`rust

file_ops.rs

```
use std::fs::File; use std::io::{Read, Write}; use std::path::Path; use encoding_rs::{UTF_8, SHIFT_JIS, EUC_JP}; use
serde::{Deserialize, Serialize};
```

```
#[derive(Serialize, Deserialize)] pub struct FileData { pub content: String, pub encoding: String, }
```

```
// パスの安全性をチェックする内部関数 fn is_path_safe(path: &str) -> bool { let p = Path::new(path); // 1. パ
スコンポーネントに "." が含まれていないかチェック if path.contains("..") { return false; } // 2. 絶対パスである
ことを推奨（Windowsの場合はドライブレターから始まるか） // ※ メモ帳なので任意の場所を開ける必要
があるが、システムディレクトリ等はOSの権限に任せる true }
```

```
#[tauri::command] pub fn read_text_file(path: String) -> Result<FileData, String> { if !is_path_safe(&path) {
return Err("不正なパスが指定されました".to_string()); }
```

```
    let mut file = File::open(&path).map_err(|e| e.to_string())?;
    let mut buffer = Vec::new();
    file.read_to_end(&mut buffer).map_err(|e| e.to_string())?;

    // 文字コードの判別（簡易版：UTF-8かそれ以外か）
    // 本格的には chardet とか使うけど、今回は encoding_rs の decode を試す
    let (res, _encoding, _has_errors) = UTF_8.decode(&buffer);
    if !_has_errors {
        return Ok(FileData {
            content: res.into_owned(),
            encoding: "UTF-8".to_string(),
        });
    }

    // UTF-8 でエラーが出たら Shift-JIS を試す
    let (res, _, _has_errors) = SHIFT_JIS.decode(&buffer);
    if !_has_errors {
        return Ok(FileData {
            content: res.into_owned(),
            encoding: "Shift-JIS".to_string(),
        });
    }

    // それでもダメなら EUC-JP
    let (res, _, _) = EUC_JP.decode(&buffer);
    Ok(FileData {
        content: res.into_owned(),
        encoding: "EUC-JP".to_string(),
    })
}
```

```
}
```

```
#[tauri::command] pub fn write_text_file(path: String, content: String, encoding: String) -> Result<(), String> { if
!is_path_safe(&path) { return Err("不正なパスが指定されました".to_string()); }
```

```
    let mut file = File::create(&path).map_err(|e| e.to_string())?;

    let bytes = match encoding.as_str() {
        "Shift-JIS" => {
            let (res, _, _) = SHIFT_JIS.encode(&content);
            res.into_owned()
        },
        "EUC-JP" => {
            let (res, _, _) = EUC_JP.encode(&content);
            res.into_owned()
        },
        _ => content.into_bytes(), // デフォルトは UTF-8
    }
```

```
};

file.write_all(&bytes).map_err(|e| e.to_string())?;
Ok(())
```

```
}
```

```
,
```

```
`rust
```

font_ops.rs

```
use std::collections::HashMap; use std::fs; use std::path::PathBuf; use serde::{Deserialize, Serialize}; use
tauri::Manager;
```

```
#[derive(Serialize, Deserialize, Clone)] pub struct SavedTab { pub name: String, pub path: Option, pub cache_id:
Option, }
```

```
#[derive(Serialize, Deserialize, Clone)] pub struct AppConfig { pub editor_font_family: String, pub
editor_font_size: u32, pub preview_font_family: String, pub preview_font_size: u32, pub theme: String, pub
last_split_ratio: f32, pub save_tabs: bool, pub saved_tabs: Vec, pub snippets: HashMap<String, String>, pub
open_external_browser: bool, pub default_encoding: String, }
```

```
impl Default for AppConfig { fn default() -> Self { let mut snippets = HashMap::new();
snippets.insert("date".to_string(), "${currentDate}".to_string());
```

```
    Self {
        editor_font_family: "system-ui".to_string(),
        editor_font_size: 16,
        preview_font_family: "system-ui".to_string(),
        preview_font_size: 16,
        theme: "vs-dark".to_string(),
        last_split_ratio: 50.0,
        save_tabs: true,
        saved_tabs: Vec::new(),
        snippets,
        open_external_browser: true,
        default_encoding: "UTF-8".to_string(),
    }
}
```

```
}
```

```
// フォントリストを実際にスキャンする内部関数 fn scan_os_fonts() -> Vec { let mut fonts = Vec::new(); let
font_dir = PathBuf::from("C:\\Windows\\Fonts");
```

```
fn scan_dir(dir: PathBuf, fonts: &mut Vec<String>) {
    if let Ok(entries) = fs::read_dir(dir) {
        for entry in entries.flatten() {
            let path = entry.path();
            if path.is_dir() {
                scan_dir(path, fonts);
            } else if let Some(ext) = path.extension() {
                let ext_str = ext.to_string_lossy().to_lowercase();
                if ext_str == "ttf" || ext_str == "otf" || ext_str == "ttc" {
                    if let Some(name) = path.file_stem() {
                        let clean_name = name.to_string_lossy().to_string();
                        let family_name = clean_name.split('-')
                            .next().unwrap_or(&clean_name);
                        fonts.push(family_name.to_string());
                    }
                }
            }
        }
    }
}

scan_dir(font_dir, &mut fonts);
fonts.sort_by(|a, b| a.to_lowercase().cmp(&b.to_lowercase()));
fonts.dedup();
fonts
```

```
}
```

```
#[tauri::command] pub fn get_os_fonts(app_handle: tauri::AppHandle) -> Vec { let config_dir =
app_handle.path().app_config_dir().unwrap_or_default(); let cache_path = config_dir.join("fonts_cache.json");
```

```
// キャッシュがあればそれを返す
if let Ok(content) = fs::read_to_string(&cache_path) {
    if let Ok(fonts) = serde_json::from_str::<Vec<String>>(&content) {
        return fonts;
    }
}

// キャッシュがなければスキャンして保存
let fonts = scan_os_fonts();
if let Ok(content) = serde_json::to_string(&fonts) {
    let _ = fs::create_dir_all(&config_dir);
    let _ = fs::write(cache_path, content);
}
fonts
```

```
}
```

```
#[tauri::command] pub fn refresh_os_fonts(app_handle: tauri::AppHandle) -> Vec { let config_dir =
app_handle.path().app_config_dir().unwrap_or_default(); let cache_path = config_dir.join("fonts_cache.json");
```

```
// 強制的に再スキャンしてキャッシュを更新
let fonts = scan_os_fonts();
if let Ok(content) = serde_json::to_string(&fonts) {
    let _ = fs::create_dir_all(&config_dir);
    let _ = fs::write(cache_path, content);
}
fonts
```

```
}
```

```
#[tauri::command] pub fn load_config(app_handle: tauri::AppHandle) -> AppConfig { let config_dir =
app_handle.path().app_config_dir().unwrap_or_default(); let config_path = config_dir.join("config.json"); if let
Ok(content) = fs::read_to_string(config_path) { if let Ok(config) = serde_json::from_str(&content) { return
config; } } AppConfig::default() }
```

```
#[tauri::command] pub fn get_config_path(app_handle: tauri::AppHandle) -> String { let config_dir =
app_handle.path().app_config_dir().unwrap_or_default();
config_dir.join("config.json").to_string_lossy().to_string() }
```

```
#[tauri::command] pub fn save_config(app_handle: tauri::AppHandle, config: AppConfig) -> Result<(), String> {
let config_dir = app_handle.path().app_config_dir().unwrap_or_default(); if !config_dir.exists() {
fs::create_dir_all(&config_dir).map_err(|e| e.to_string())?; } let config_path = config_dir.join("config.json"); let
content = serde_json::to_string_pretty(&config).map_err(|e| e.to_string())?; fs::write(config_path,
content).map_err(|e| e.to_string())?; Ok(()) }
```

```
// cache_id のサニタイズ（英数字とハイフンのみ許可） fn sanitize_cache_id(id: &str) -> String {
id.chars().filter(|c| c.is_alphanumeric() || *c == '-').collect() }
```

```
#[tauri::command] pub fn save_tab_cache(app_handle: tauri::AppHandle, cache_id: String, content: String) ->
Result<(), String> { let safe_id = sanitize_cache_id(&cache_id); let cache_dir =
app_handle.path().app_cache_dir().unwrap_or_default().join("tab_cache"); if !cache_dir.exists() {
fs::create_dir_all(&cache_dir).map_err(|e| e.to_string())?; } let cache_path = cache_dir.join(format!("{}", safe_id)); fs::write(cache_path, content).map_err(|e| e.to_string())?; Ok(()) }
```

```
#[tauri::command] pub fn load_tab_cache(app_handle: tauri::AppHandle, cache_id: String) -> Result<String,
String> { let safe_id = sanitize_cache_id(&cache_id); let cache_path =
app_handle.path().app_cache_dir().unwrap_or_default().join("tab_cache").join(format!("{}", safe_id));
fs::read_to_string(cache_path).map_err(|e| e.to_string()) }
```

```
,
```

```
`typescript
```

main.tsx

```
import React from "react"; import ReactDOM from "react-dom/client"; import App from "./App";

console.log("React app is starting...");

const rootElement = document.getElementById("root");

if (rootElement) { try { ReactDOM.createRoot(rootElement).render( <React.StrictMode> </React.StrictMode> );
console.log("React mount requested."); } catch (error) { console.error("React render failed:", error); } } else {
console.error("Root element not found!"); }

、

`typescript
```

App.tsx

```
import { useState, useEffect, useRef, useCallback, useMemo } from "react"; import { invoke } from '@tauri-
apps/api/core'; import { open, save } from '@tauri-apps/plugin-dialog'; import './App.css'; import
MonacoEditor from './components/Editor'; import TabBar from './components/TabBar'; import Preview from
'./components/Preview'; import Settings from './components/Settings'; import StatusBar from
'./components/StatusBar'; import CommandPalette from './components/CommandPalette'; import {
useConfig } from './hooks/useConfig'; import Eye from "lucide-react/dist/esm/icons/eye"; import Edit3 from
"lucide-react/dist/esm/icons/edit-3"; import Columns from "lucide-react/dist/esm/icons/columns"; import
SettingsIcon from "lucide-react/dist/esm/icons/settings"; import FilePlus from "lucide-
react/dist/esm/icons/file-plus"; import FolderOpen from "lucide-react/dist/esm/icons/folder-open"; import
Save from "lucide-react/dist/esm/icons/save"; import Search from "lucide-react/dist/esm/icons/search";
import Repeat from "lucide-react/dist/esm/icons/repeat"; import Undo2 from "lucide-
react/dist/esm/icons/undo-2"; import Redo2 from "lucide-react/dist/esm/icons/redo-2"; import LinkIcon from
"lucide-react/dist/esm/icons/link"; import Link2Off from "lucide-react/dist/esm/icons/link-2-off";

import { useTabs, Tab } from './hooks/useTabs'; import { useAutoSave } from './hooks/useAutoSave'; import {
useCommandPalette, Command } from './hooks/useCommandPalette';

type ViewMode = "editor" | "preview" | "split"; const generateId = () =>
Math.random().toString(36).substring(2, 11) + Date.now().toString(36);

const translations = { ja: { new: "新規", open: "開く", save: "保存", undo: "元に戻す", redo: "やり直し", search:
"検索", replace: "置換", loading: "読み込み中...", openSettings: "設定を開く", syncOn: "スクロール同期: ON",
syncOff: "スクロール同期: OFF" }, en: { new: "New", open: "Open", save: "Save", undo: "Undo", redo: "Redo",
search: "Search", replace: "Replace", loading: "Loading...", openSettings: "Open Settings", syncOn: "Scroll Sync:
ON", syncOff: "Scroll Sync: OFF" } };

function App() { // 1. 全ての Hooks は一番最初、かつ無条件に呼ぶ const { config, setConfig, isLoading:
isLoadingConfig } = useConfig(); const { tabs, activeTabId, activeTab, setActiveTabId, updateTabContent, addTab,
closeTab, openFile, saveFile, setAllTabs, renameTab, updateTabEncoding, setTabs } = useTabs(); const { isOpen:
isCommandPaletteOpen, initialSearch, closePalette, togglePalette } = useCommandPalette();

const [viewMode, setViewMode] = useState("split"); const [syncScroll, setSyncScroll] = useState(true); const
[showSettings, setShowSettings] = useState(false); const [sidebarWidth, setSidebarWidth] = useState(200);
```

```
const [splitRatio, setSplitRatio] = useState(50); const [resizingType, setResizingType] = useState<"sidebar" | "split" | null>(null);
```

```
const editorRef = useRef(null); const previewRef = useRef(null); const isSyncing = useRef(false); const [isAppInitialized, setIsAppInitialized] = useState(false);
```

```
const t = translations[config?.lang || 'ja'];
```

```
// 2. コールバック群の定義 const handleOpenFile = useCallback(async () => { try { const selected = await open({ multiple: false }); if (selected && typeof selected === 'string') openFile(selected); } catch (e) {} }, [openFile]);
```

```
const handleSaveAs = useCallback(async () => { if (!activeTabId) return; const tab = tabs.find(t => t.id === activeTabId); if (!tab) return; try { const selected = await save({ defaultPath: tab.name }); if (selected) await saveFile(activeTabId, selected); } catch (e) { console.error("Save As failed", e); } }, [tabs, activeTabId, saveFile]);
```

```
const handleSave = useCallback(async () => { if (!activeTabId) return; const tab = tabs.find(t => t.id === activeTabId); if (!tab || tab.id === 'initial') return;
```

```
    if (tab.path) {
      await saveFile(activeTabId);
    } else {
      await handleSaveAs();
    }
  }
```

```
}, [tabs, activeTabId, saveFile, handleSaveAs]);
```

```
const handleOpenConfigJson = useCallback(async () => { try { const configPath = await invoke('get_config_path'); await openFile(configPath); setShowSettings(false); } catch (e) { console.error("Failed to open config JSON", e); } }, [openFile]);
```

```
const appCommands = useMemo<Command[]>(() => { if (!config) return []; return [ { id: 'new-file', name: t.new, category: 'File', shortcut: 'Ctrl+N', action: () => addTab('Untitled', '', null, config.default_encoding) }, { id: 'open-file', name: t.open, category: 'File', shortcut: 'Ctrl+O', action: handleOpenFile }, { id: 'save-file', name: t.save, category: 'File', shortcut: 'Ctrl+S', action: handleSave }, { id: 'open-settings', name: t.openSettings, category: 'App', shortcut: 'Ctrl+,', action: () => setShowSettings(true) }, { id: 'open-settings-json', name: 'Settings: Open JSON', category: 'App', action: handleOpenConfigJson }, { id: 'view-editor', name: '表示: エディタのみ', category: 'View', action: () => setViewMode('editor') }, { id: 'view-split', name: '表示: 分割画面', category: 'View', action: () => setViewMode('split') }, { id: 'view-preview', name: '表示: プレビューのみ', category: 'View', action: () => setViewMode('preview') }, { id: 'toggle-sync', name: 'Scroll Sync: ' + (syncScroll ? 'ON' : 'OFF'), category: 'View', action: () => setSyncScroll(!syncScroll) }, { id: 'enc-utf8', name: 'Encoding: UTF-8', category: 'Encoding', action: () => activeTabId && updateTabEncoding(activeTabId, 'UTF-8') }, { id: 'enc-sjis', name: 'Encoding: Shift-JIS', category: 'Encoding', action: () => activeTabId && updateTabEncoding(activeTabId, 'Shift-JIS') }, { id: 'enc-eucjp', name: 'Encoding: EUC-JP', category: 'Encoding', action: () => activeTabId && updateTabEncoding(activeTabId, 'EUC-JP') }, { id: 'lang-md', name: 'Language: Markdown', category: 'Language', action: () => activeTabId && setTabs((prev: Tab[]) => prev.map(t => t.id === activeTabId ? { ...t, language: 'markdown' } : t)) }, { id: 'lang-js', name: 'Language: JavaScript', category: 'Language', action: () => activeTabId && setTabs((prev: Tab[]) => prev.map(t => t.id === activeTabId ? { ...t, language: 'javascript' } : t)) },
```

```
{ id: 'lang-ts', name: 'Language: TypeScript', category: 'Language', action: () => activeTabId && setTabs((prev: Tab[]) => prev.map(t => t.id === activeTabId ? { ...t, language: 'typescript' } : t)) }, { id: 'lang-rust', name: 'Language: Rust', category: 'Language', action: () => activeTabId && setTabs((prev: Tab[]) => prev.map(t => t.id === activeTabId ? { ...t, language: 'rust' } : t)) }, { id: 'lang-json', name: 'Language: JSON', category: 'Language', action: () => activeTabId && setTabs((prev: Tab[]) => prev.map(t => t.id === activeTabId ? { ...t, language: 'json' } : t)) }, ], [config, t, addTab, handleOpenFile, handleSave, handleOpenConfigJson, activeTabId, updateTabEncoding, syncScroll, setTabs]);
```

```
// 3. 画面初期化の Effect useEffect() => { if (!isConfigLoaded || !config || isAppInitialized) return;
```

```
const initTabs = async () => {
  if (config.save_tabs && config.saved_tabs && config.saved_tabs.length > 0) {
    const restoredTabs: Tab[] = [];
    for (const st of config.saved_tabs) {
      if (!st) continue;
      try {
        let content = '';
        let encoding = 'UTF-8';
        if (st.cache_id) content = await invoke<string>('load_tab_cache', {
          cacheId: st.cache_id }).catch(() => '');
        if (!content && st.path) {
          const data: any = await invoke('read_text_file', { path: st.path
        }).catch(() => null);
          if (data) { content = data.content; encoding = data.encoding; }
        }
        restoredTabs.push({
          id: generateId(), name: st.name || 'Untitled', path: st.path || null,
          content: content || '', encoding,
          isModified: false, isCustomName: !!st.path, cacheId: st.cache_id ||
          generateId(),
          language: st.path ? (st.name.split('.').pop() || 'markdown') :
          'markdown'
        });
      } catch (e) {}
    }
    if (restoredTabs.length > 0) setAllTabs(restoredTabs);
  }
  setIsAppInitialized(true);
};
initTabs();
```

```
}, [config, isConfigLoaded, isAppInitialized, setAllTabs]);
```

```
// 4. 設定の保存 Effect useEffect() => { if (!isAppInitialized || !config) return; const saved_tabs =
config.save_tabs ? tabs.map(t => ({ name: t.name, path: t.path, cache_id: t.cacheId || null })): [];
```

```
if (JSON.stringify(saved_tabs) !== JSON.stringify(config.saved_tabs) || splitRatio
  !== config.last_split_ratio) {
```

```
    setConfig({ saved_tabs, last_split_ratio: splitRatio });
  }
```

```
}, [tabs, splitRatio, config, isApplnInitialized, setConfig]);
```

```
// 5. キーボードショートカット useEffect(() => { const handleKeyDown = (e: KeyboardEvent) => { if (e.ctrlKey
&& e.shiftKey && e.key === 'P') { e.preventDefault(); e.stopPropagation(); togglePalette(); } else if (e.ctrlKey
&& e.key === 'n') { e.preventDefault(); addTab('Untitled', '', null, config.default_encoding); } else if (e.ctrlKey
&& e.key === 'o') { e.preventDefault(); handleOpenFile(); } else if (e.ctrlKey && e.key === 's') {
e.preventDefault(); handleSave(); } else if (e.ctrlKey && e.key === ',') { e.preventDefault();
setShowSettings(true); } }; window.addEventListener('keydown', handleKeyDown, { capture: true }); return () =>
window.removeEventListener('keydown', handleKeyDown, { capture: true }); }, [togglePalette, addTab,
handleOpenFile, handleSave, config?.default_encoding]);
```

```
// 6. 自動保存 Hook useAutoSave(activeTab || tabs[0] || { id: 'initial', path: null, isModified: false, content: '',
cacheld: 'initial' }, saveFile);
```

```
// 7. レイアウト調整 useEffect(() => { if (editorRef.current) editorRef.current.layout(); }, [sidebarWidth,
splitRatio, viewMode]);
```

```
const handleMouseMove = useCallback((e: MouseEvent) => { if (!resizingType) return; if (resizingType ===
"sidebar") { if (e.clientX > 50 && e.clientX < 600) setSidebarWidth(e.clientX); } else if (resizingType === "split")
{ const container = document.getElementById('editor-preview-container'); if (container) { const rect =
container.getBoundingClientRect(); const newRatio = ((e.clientX - rect.left) / rect.width) * 100; if (newRatio > 5
&& newRatio < 95) setSplitRatio(newRatio); } } }, [resizingType]);
```

```
useEffect(() => { const handleMouseUp = () => setResizingType(null); window.addEventListener('mousemove',
handleMouseMove); window.addEventListener('mouseup', handleMouseUp); return () => {
window.removeEventListener('mousemove', handleMouseMove); window.removeEventListener('mouseup',
handleMouseUp); } }, [handleMouseMove]);
```

```
// 8. スクロール同期のハンドラ (handleScroll ではなく明確な名前に統一) const handleEditorScroll =
useCallback((top: number, total: number, startLine?: number) => { if (!syncScroll || isSyncing.current ||
viewMode !== "split") return;
```

```
    if (previewRef.current) {
      const previewEl = previewRef.current.querySelector('.markdown-body') as
      HTMLElement;
      if (previewEl) {
        // 0. 最下部への到達を検知して強制同期
        if (total > 0 && top >= total - 5) {
          isSyncing.current = true;
          const previewTotal = previewEl.scrollHeight - previewEl.clientHeight;
          previewEl.scrollTop = previewTotal;
          setTimeout(() => { isSyncing.current = false; }, 50);
          return;
        }
      }

      isSyncing.current = true;
    }
  }
```

```

// 1. 行番号ベースの同期（線形補間付き）
if (startLine !== undefined) {
  const elements = Array.from(previewEl.querySelectorAll('[data-line]'));

  if (elements.length > 0) {
    let prevEl: HTMLElement | null = null;
    let nextEl: HTMLElement | null = null;

    for (const el of elements) {
      const line = parseInt((el as HTMLElement).dataset.line || "0");
      if (line <= startLine) {
        prevEl = el as HTMLElement;
      } else {
        nextEl = el as HTMLElement;
        break;
      }
    }

    if (prevEl) {
      const prevLine = parseInt(prevEl.dataset.line || "0");
      const prevTop = prevEl.offsetTop - previewEl.offsetTop;

      if (nextEl) {
        const nextLine = parseInt(nextEl.dataset.line || "0");
        const nextTop = nextEl.offsetTop - previewEl.offsetTop;

        // 行数に応じた割合を計算（線形補間）
        const ratio = (startLine - prevLine) / (nextLine - prevLine);
        previewEl.scrollTop = prevTop + (nextTop - prevTop) * ratio - 20;
      } else {
        previewEl.scrollTop = prevTop - 20;
      }

      setTimeout(() => { isSyncing.current = false; }, 50);
      return;
    }
  }
}

// 2. フォールバック：パーセントベースの同期
const previewTotal = previewEl.scrollHeight - previewEl.clientHeight;
previewEl.scrollTop = previewTotal * (total > 0 ? top / total : 0);
}
}

setTimeout(() => { isSyncing.current = false; }, 50);

```

```

}, [viewMode, syncScroll]);

```

```

const handlePreviewScroll = useCallback((top: number, total: number) => { if (!syncScroll || isSyncing.current || viewMode !== "split") return;

```

```
if (editorRef.current && previewRef.current) {
  const previewEl = previewRef.current.querySelector('.markdown-body') as
  HTMLElement;
  if (previewEl) {
    // 0. 最下部への到達を検知
    if (total > 0 && top >= total - 5) {
      isSyncing.current = true;
      const editorTotal = editorRef.current.getScrollHeight() -
      editorRef.current.getLayoutInfo().height;
      editorRef.current.setScrollTop(editorTotal);
      setTimeout(() => { isSyncing.current = false; }, 50);
      return;
    }

    isSyncing.current = true;
    // 1. 逆・行番号ベースの同期（プレビュー位置から行番号を逆算）
    const elements = Array.from(previewEl.querySelectorAll('[data-line]'));

    if (elements.length > 0) {
      let prevEl: HTMLElement | null = null;
      let nextEl: HTMLElement | null = null;

      // 現在のスクロール位置（top）に近い要素を探す
      for (const el of elements) {
        const elTop = (el as HTMLElement).offsetTop - previewEl.offsetTop;
        if (elTop <= top + 30) { // 少し余裕を持たせる
          prevEl = el as HTMLElement;
        } else {
          nextEl = el as HTMLElement;
          break;
        }
      }

      if (prevEl) {
        const prevLine = parseInt(prevEl.dataset.line || "1");
        const prevTop = prevEl.offsetTop - previewEl.offsetTop;
        let targetLine = prevLine;

        if (nextEl) {
          const nextLine = parseInt(nextEl.dataset.line || String(prevLine + 1));
          const nextTop = nextEl.offsetTop - previewEl.offsetTop;
          const ratio = (top + 30 - prevTop) / (nextTop - prevTop);
          targetLine = prevLine + (nextLine - prevLine) * Math.max(0, Math.min(1,
ratio));
        }

        // Monacoのエディタの該当行のピクセル位置へスクロール

        editorRef.current.setScrollTop(editorRef.current.getTopForLineNumber(Math.floor(targetLine)));
        setTimeout(() => { isSyncing.current = false; }, 50);
        return;
      }
    }
  }
}
```

```

    }
  }

  // 2. フォールバック: パーセントベースの同期
  const editorTotal = editorRef.current.getScrollHeight() -
    editorRef.current.getLayoutInfo().height;
  editorRef.current.setScrollTop(editorTotal * (total > 0 ? top / total : 0));
}
}
setTimeout(() => { isSyncing.current = false; }, 50);

```

```
}, [viewMode, syncScroll]);
```

```
// テーマを body に反映させる (Rules of Hooks により早期リターンの前に書く!) useEffect(() => { if
(config?.theme) { document.body.setAttribute('data-theme', config.theme); } }, [config?.theme]);
```

```
// 9. レンダリング前の最終チェック if (!isConfigLoaded || !config) return <div style={{ backgroundColor:
'#1e1e1e', color: '#666', height: '100vh', display: 'flex', alignItems: 'center', justifyContent: 'center' }}>
{translations.ja.loading}; const currentTab = activeTab || tabs[0] || { id: 'temp', name: 'Untitled', content: '',
language: 'plaintext', encoding: 'UTF-8' }; const charCount = currentTab.content.length;
```

```
return ( <div className="app-wrapper" style={{ display: 'flex', flexDirection: 'column', height: '100vh',
overflow: 'hidden' }}> <div className="container" style={{ flexGrow: 1, display: "flex", backgroundColor: 'var(-
-bg-main)', color: 'var(--text-main)', cursor: resizingType ? 'col-resize' : 'default', userSelect: resizingType ?
'none' : 'auto', overflow: 'hidden' }}> <div style={{ width: `${sidebarWidth}px`, flexShrink: 0, height: '100%',
minWidth: '50px', display: 'flex' }}> <TabBar tabs={tabs} activeTabId={activeTabId} onTabClick={setActiveTabId}
onTabClose={closeTab} onNewTab={() => addTab('Untitled', '', null, config.default_encoding)} onRenameTab=
{renameTab} /> <div onMouseDown={(e) => { setResizingType("sidebar"); e.preventDefault(); }} style={{ width:
'4px', height: '100%', backgroundColor: resizingType === "sidebar" ? 'var(--primary)' : 'var(--border-main)',
cursor: 'col-resize' }} />
```

```

    <div style={{ flexGrow: 1, display: 'flex', flexDirection: 'column', overflow:
'hidden', minWidth: 0 }}>
      <div style={{ display: 'flex', alignItems: 'center', backgroundColor: 'var(-
-bg-actionbar)', padding: '4px 10px', borderBottom: '1px solid var(--border-
main)', fontSize: '12px', zIndex: 1000, flexShrink: 0 }}>
        <div style={{ display: 'flex', gap: '15px', marginRight: '20px' }}>
          <div onClick={() => addTab('Untitled', '', null,
config.default_encoding)} style={{ cursor: 'pointer', display: 'flex', alignItems:
'center', gap: '4px' }}><FilePlus size={14} /> {t.new}</div>
          <div onClick={handleOpenFile} style={{ cursor: 'pointer', display:
'flex', alignItems: 'center', gap: '4px' }}><FolderOpen size={14} /> {t.open}
</div>
          <div onClick={handleSave} style={{ cursor: 'pointer', display: 'flex',
alignItems: 'center', gap: '4px' }}><Save size={14} /> {t.save}</div>
        </div>
        <div style={{ display: 'flex', gap: '10px' }}>
          <button onClick={() => editorRef.current?.trigger('menubar', 'undo',
null)} style={{ background: 'none', border: 'none', color: "var(--text-muted)",
cursor: 'pointer', padding: 2 }} title={t.undo}><Undo2 size={18} /></button>

```

```

        <button onClick={() => editorRef.current?.trigger('menubar', 'redo',
null)} style={{ background: 'none', border: 'none', color: "var(--text-muted)",
cursor: 'pointer', padding: 2 }} title={t.redo}><Redo2 size={18} /></button>
    </div>
    <div style={{ flexGrow: 1 }} />
    <div style={{ display: 'flex', gap: '8px', alignItems: 'center' }}>
        <button onClick={() => { editorRef.current?.focus();
editorRef.current?.trigger('menubar', 'actions.find', null); }} style={{
background: 'none', border: 'none', color: "var(--text-muted)", cursor: 'pointer',
padding: 4 }} title={t.search}><Search size={18} /></button>
        <button onClick={() => { editorRef.current?.focus();
editorRef.current?.trigger('menubar', 'editor.action.startFindReplaceAction',
null); }} style={{ background: 'none', border: 'none', color: "var(--text-muted)",
cursor: 'pointer', padding: 4 }} title={t.replace}><Repeat size={18} /></button>
        <div style={{ width: '1px', height: '20px', backgroundColor: 'var(--
border-main)', margin: '0 4px' }} />

        <button onClick={() => setSyncScroll(!syncScroll)} style={{ background:
'none', border: 'none', color: syncScroll ? "var(--primary)" : "var(--text-
muted)", cursor: 'pointer', padding: 4 }} title={syncScroll ? "Sync Scroll: ON" :
"Sync Scroll: OFF"}>
            {syncScroll ? <LinkIcon size={18} /> : <Link20ff size={18} />}
        </button>

        <div style={{ width: '1px', height: '20px', backgroundColor: 'var(--
border-main)', margin: '0 4px' }} />
        <button onClick={() => setViewMode("editor")} style={{ background:
'none', border: 'none', color: viewMode === "editor" ? "var(--primary)" : "var(--
text-muted)", cursor: 'pointer', padding: 4 }}><Edit3 size={18} /></button>
        <button onClick={() => setViewMode("split")} style={{ background:
'none', border: 'none', color: viewMode === "split" ? "var(--primary)" : "var(--
text-muted)", cursor: 'pointer', padding: 4 }}><Columns size={18} /></button>
        <button onClick={() => setViewMode("preview")} style={{ background:
'none', border: 'none', color: viewMode === "preview" ? "var(--primary)" : "var(--
text-muted)", cursor: 'pointer', padding: 4 }}><Eye size={18} /></button>
        <div style={{ width: '1px', height: '20px', backgroundColor: 'var(--
border-main)', margin: '0 4px' }} />
        <button onClick={() => setShowSettings(true)} style={{ background:
'none', border: 'none', color: "var(--text-muted)", cursor: 'pointer', padding: 4
}}><SettingsIcon size={18} /></button>
    </div>
</div>

<div id="editor-preview-container" style={{ flexGrow: 1, display: 'flex',
overflow: 'hidden', position: 'relative' }}>
    {(viewMode === "editor" || viewMode === "split") && (
        <div style={{ width: viewMode === "split" ? `${splitRatio}%` : '100%',
height: '100%', flexShrink: 0, minWidth: 0 }}>
            <MonacoEditor key={activeTabId} value={currentTab.content} onChange=
{(v) => updateTabContent(activeTabId, v || "")} fontFamily=
{config.editor_font_family} fontSize={config.editor_font_size} theme=
{config.theme} language={currentTab.language} onMount={(editor) => {
editorRef.current = editor; }} onScroll={handleEditorScroll} snippets=
{config.snippets} />

```

```

        </div>
      )}
      {viewMode === "split" && (
        <div onMouseDown={(e) => { setResizingType("split"); e.preventDefault();
      }} style={{ width: '4px', height: '100%', backgroundColor: resizingType ===
      "split" ? 'var(--primary)' : 'var(--border-main)', cursor: 'col-resize', zIndex:
      10, flexShrink: 0 }} />
      )}
      {(viewMode === "preview" || viewMode === "split") && (
        <div ref={previewRef} style={{ flexGrow: 1, height: '100%', overflow:
      'hidden', minWidth: 0 }}>
          <Preview content={currentTab.content} fontFamily=
      {config.preview_font_family} fontSize={config.preview_font_size} theme=
      {config.theme} openExternalBrowser={config.open_external_browser} onScroll=
      {handlePreviewScroll} />
        </div>
      )}
    </div>
  </div>
</div>
{showSettings && <Settings config={config} onSave={(newCfg) =>
setConfig(newCfg)} onClose={() => setShowSettings(false)} onOpenJson=
{handleOpenConfigJson} />}
<StatusBar
  charCount={charCount}
  language={currentTab.language || 'plaintext'}
  encoding={currentTab.encoding || 'UTF-8'}
  theme={config.theme}
  fontFamily={config.editor_font_family}
  onEncodingClick={() => togglePalette('Encoding: ')}
  onLanguageClick={() => togglePalette('Language: ')}
/>
<CommandPalette
  isOpen={isCommandPaletteOpen}
  initialSearch={initialSearch}
  commands={appCommands}
  onExecute={(cmd) => { cmd.action(); closePalette(); }}
  onClose={closePalette}
  theme={config.theme}
/>
</div>

```

```
);}
```

```
export default App;
```

```
,
```

```
`typescript
```

useAutoSave.ts

```
import { useEffect, useRef } from 'react'; import { invoke } from '@tauri-apps/api/core'; import { Tab } from './useTabs';
```

```
/**
```

- 自動保存とタブキャッシュの保存を統合したフック。
- タイピングが止まったタイミングでまとめて保存することで、パフォーマンスを向上させ、
- ReactのState更新とMonacoの競合を回避します。 */ export const useAutoSave = (activeTab: Tab, onSave: (id: string) => Promise) => { const timerRef = useRef<ReturnType | null>(null);

```
useEffect(() => { // 古いタイマーをクリア if (timerRef.current) clearTimeout(timerRef.current);
```

```
// タイピング中（中身が空でない場合）はデバウンスして保存
timerRef.current = setTimeout(async () => {
  try {
    // 1. キャッシュ保存（常に実行）
    if (activeTab.cacheId && activeTab.content !== undefined) {
      await invoke('save_tab_cache', { cacheId: activeTab.cacheId, content:
activeTab.content });
    }

    // 2. 自動保存（ファイルパスがあり、変更がある場合のみ）
    if (activeTab.path && activeTab.isModified) {
      await onSave(activeTab.id);
    }
  } catch (e) {
    console.error('AutoSave failed:', e);
  }
}, 1500); // 1.5秒入力が止まったら保存

return () => {
  if (timerRef.current) clearTimeout(timerRef.current);
};
```

```
}, [activeTab.content, activeTab.path, activeTab.isModified, activeTab.id, activeTab.cacheId, onSave]);
```

```
,
```

```
`typescript
```

useCommandPalette.ts

```
import { useState, useCallback } from 'react';
```

```
export interface Command { id: string; name: string; category?: string; shortcut?: string; action: () => void; }
```

```
/**
```

- コマンドパレットの表示状態のみを管理するシンプルなフック。

- コマンドのリスト管理を呼び出し側に移譲することで、無限ループを回避します。 */ export function useCommandPalette() { const [isOpen, setIsOpen] = useState(false); const [initialSearch, setInitialSearch] = useState("");

```
const openPalette = useCallback((search: string = "") => { setInitialSearch(search); setIsOpen(true); }, []); const
closePalette = useCallback(() => setIsOpen(false), []); const togglePalette = useCallback((search: string = "") =>
{ setIsOpen(prev => { if (!prev) setInitialSearch(search); return !prev; }); }, []);
```

```
return { isOpen, initialSearch, openPalette, closePalette, togglePalette }; }
```

```
,
```

```
`typescript
```

useConfig.ts

```
import { useState, useEffect, useCallback } from 'react'; import { invoke } from '@tauri-apps/api/core';
```

```
export interface SavedTab { name: string; path: string | null; cache_id: string | null; }
```

```
export interface AppConfig { editor_font_family: string; editor_font_size: number; preview_font_family: string;
preview_font_size: number; theme: 'vs-dark' | 'light'; save_tabs: boolean; last_split_ratio: number; lang: 'ja' | 'en';
saved_tabs?: SavedTab[]; snippets: Record<string, string>; open_external_browser: boolean; default_encoding:
string; }
```

```
export const defaultConfig: AppConfig = { editor_font_family: 'system-ui', editor_font_size: 16,
preview_font_family: 'system-ui', preview_font_size: 16, theme: 'vs-dark', save_tabs: true, last_split_ratio: 50,
lang: 'ja', saved_tabs: [], snippets: { 'date': '${currentDate}' }, open_external_browser: true, default_encoding:
'UTF-8' };
```

```
export function useConfig() { const [config, setConfigState] = useState(defaultConfig); const [isLoading,
setIsLoaded] = useState(false);
```

```
const loadConfig = useCallback(async () => { try { const loadedConfig = await invoke('load_config'); if
(loadedConfig) { setConfigState(prev => ({ ...prev, ...loadedConfig })); } } catch (e) { console.error('Failed to load
config', e); } finally { setIsLoaded(true); } }, []);
```

```
useEffect(() => { loadConfig(); }, [loadConfig]);
```

```
const setConfig = useCallback(async (newConfig: Partial) => { setConfigState(prev => { const updatedConfig
= { ...prev, ...newConfig }; invoke('save_config', { config: updatedConfig }).catch(e => { console.error('Failed to
save config', e); }); return updatedConfig; }); }, []);
```

```
return { config, setConfig, isLoading, reloadConfig: loadConfig }; }
```

```
,
```

```
`typescript
```

useTabs.ts

```
import { useState, useCallback } from 'react'; import { invoke } from '@tauri-apps/api/core';

export interface Tab { id: string; name: string; path: string | null; content: string; encoding: string; isModified:
boolean; isCustomName?: boolean; language?: string; // 言語設定 cached?: string; // キャッシュ用ID }

// 拡張子と言語のマップ const extToLang: { [key: string]: string } = { 'js': 'javascript', 'ts': 'typescript', 'tsx':
'typescript', 'jsx': 'javascript', 'py': 'python', 'rs': 'rust', 'html': 'html', 'css': 'css', 'json': 'json', 'md': 'markdown',
'txt': 'plaintext', 'sql': 'sql', 'yaml': 'yaml', 'yml': 'yaml', 'sh': 'shell', 'bat': 'powershell', 'ps1': 'powershell', };

export const useTabs = () => { const [tabs, setTabs] = useState<Tab[]>([ { id: 'initial', name: 'Untitled', path:
null, content: '', encoding: 'UTF-8', isModified: false, isCustomName: false, language: 'markdown', cached:
'initial' } ]); const [activeTabId, setActiveTabId] = useState('initial');

const activeTab = tabs.find(t => t.id === activeTabId) || tabs[0];

const getLanguageFromPath = (pathOrName: string) => { const fileName = pathOrName.split(/[\/]/).pop() ||
pathOrName; const ext = fileName.split('.').pop()?.toLowerCase(); return (ext && extToLang[ext]) || 'plaintext'; };

const updateTabContent = useCallback((id: string, content: string) => { setTabs(prev => prev.map(t => { if (t.id
=== id) { let newName = t.name; // 【修正】ユーザーが名前を付けてない (isCustomName: false) かつ、現在
の名前が空か Untitled の時だけ自動ネーミング ! if (!t.isCustomName && (t.name === 'Untitled' || t.name
=== '')) { const firstContentLine = content.split("\n").find(line => line.trim().length > 0);
if (firstContentLine) { newName = firstContentLine.trim().substring(0, 20); } }
return { ...t, content, name: newName, isModified: true };
}
return t;
}));
}, []);
```

```
return { ...t, content, name: newName, isModified: true };
}
return t;
}));
}, []);
```

```
const addTab = useCallback((name: string, content: string = '', path: string | null = null, encoding: string =
'UTF-8', isCustomName: boolean = false) => { const id = crypto.randomUUID(); const cached =
crypto.randomUUID(); const language = getLanguageFromPath(name); // 最初から名前がある場合はカスタム
名として扱う const finallyCustomName = isCustomName || (name !== 'Untitled' && name !== ''); const
newTab: Tab = { id, name: name || 'Untitled', path, content, encoding, isModified: false, isCustomName:
finallyCustomName, language, cached };

// 新規タブ作成時もキャッシュ保存
invoke('save_tab_cache', { cacheId, content }).catch(console.error);

setTabs(prev => [...prev, newTab]);
setActiveTabId(id);
return id;
```

```
, []);
```

```
const closeTab = useCallback((id: string) => { setTabs(prev => { if (prev.length === 1) return prev; const
newTabs = prev.filter(t => t.id !== id); if (activeTabId === id) { setActiveTabId(newTabs[newTabs.length - 1].id);
} return newTabs; }); }, [activeTabId]);
```

```
const openFile = async (path: string) => { try { const data: { content: string, encoding: string } = await
invoke('read_text_file', { path }); const name = path.split(/[\\\/]/).pop() || 'Untitled'; const existingTab = tabs.find(t
=> t.path === path); if (existingTab) { setActiveTabId(existingTab.id); return; } addTab(name, data.content, path,
data.encoding, true); } catch (e) { console.error(e); alert('ファイルを開けなかったよ 🤖\n' + e); } };
```

```
const saveFile = async (id: string, customPath?: string) => { const tab = tabs.find(t => t.id === id); if (!tab)
return; const targetPath = customPath || tab.path; if (!targetPath) return;
```

```
try {
  await invoke('write_text_file', { path: targetPath, content: tab.content,
encoding: tab.encoding });
  const name = targetPath.split(/[\\\/]/).pop() || tab.name;
  const language = getLanguageFromPath(name);
  // 保存したらカスタム名確定！
  setTabs(prev => prev.map(t => t.id === id ? { ...t, path: targetPath, name,
isModified: false, isCustomName: true, language } : t));
} catch (e) {
  console.error(e);
  alert('保存に失敗しちゃった 🤖\n' + e);
}
```

```
};
```

```
const setAllTabs = useCallback((newTabs: Tab[]) => { setTabs(newTabs); if (newTabs.length > 0)
setActiveTabId(newTabs[0].id); }, []);
```

```
const renameTab = useCallback((id: string, newName: string) => { const language =
getLanguageFromPath(newName); // 手動リネームはカスタム名確定！ setTabs(prev => prev.map(t => t.id
=== id ? { ...t, name: newName, isCustomName: true, language } : t)); }, []);
```

```
const updateTabEncoding = useCallback((id: string, encoding: string) => { setTabs(prev => prev.map(t => t.id
=== id ? { ...t, encoding, isModified: true } : t)); }, []);
```

```
return { tabs, activeTabId, activeTab, setActiveTabId, updateTabContent, addTab, closeTab, openFile, saveFile,
setTabs, setAllTabs, renameTab, updateTabEncoding }; };
```

```
、
```

```
`typescript
```

vite.config.ts

```
import { defineConfig } from "vite"; import react from "@vitejs/plugin-react";
```

```
// @ts-expect-error process is a nodejs global const host = process.env.TAURI_DEV_HOST;
```

```
export default defineConfig(async () => ({ plugins: [react()], clearScreen: false, server: { port: 1420, strictPort: true, host: host || false, hmr: host ? { protocol: "ws", host, port: 1421 } : undefined, watch: { ignored: ["/src-tauri/"] }, }, // Tauri のビルド用設定を追加！ envPrefix: ['VITE_', 'TAURI_'], build: { target: process.env.TAURI_ENV === 'windows' ? 'chrome105' : 'safari13', minify: false, cssMinify: false, sourcemap: !!process.env.TAURI_DEBUG, }, optimizeDeps: { include: ['@monaco-editor/react', 'lucide-react', 'markdown-it', 'dompurify'], }, }));
```

```
`json
```

package.json

```
{ "name": "tauri-app", "private": true, "version": "0.1.0", "type": "module", "scripts": { "dev": "vite", "build": "node --stack-size=10000 ./node_modules/vite/bin/vite.js build", "preview": "vite preview", "tauri": "tauri" }, "dependencies": { "@monaco-editor/react": "^4.7.0", "@tauri-apps/api": "^2", "@tauri-apps/plugin-dialog": "^2.7.0", "@tauri-apps/plugin-opener": "^2", "@tauri-apps/plugin-shell": "^2.3.5", "dompurify": "^3.4.1", "github-markdown-css": "^5.9.0", "highlight.js": "^11.11.1", "lucide-react": "^1.8.0", "markdown-it": "^14.1.1", "markdown-it-anchor": "^9.2.0", "react": "^19.1.0", "react-dom": "^19.1.0" }, "devDependencies": { "@tauri-apps/cli": "^2", "@types/dompurify": "^3.0.5", "@types/highlight.js": "^9.12.4", "@types/markdown-it": "^14.1.2", "@types/markdown-it-anchor": "^4.0.4", "@types/react": "^19.1.8", "@types/react-dom": "^19.1.6", "@vitejs/plugin-react": "^4.6.0", "typescript": "~5.8.3", "vite": "^6.2.0" }
```

```
`json
```

tauri.conf.json

```
{ "$schema": "https://schema.tauri.app/config/2", "productName": "モナコノート", "version": "0.1.0", "identifier": "com.monaco-note.app", "build": { "beforeDevCommand": "npm run dev", "devUrl": "http://localhost:1420", "beforeBuildCommand": "", "frontendDist": "../dist" }, "app": { "windows": [ { "title": "モナコノート", "width": 1000, "height": 800 } ], "security": { "csp": null } }, "bundle": { "active": true, "targets": "all", "icon": [ "icons/32x32.png", "icons/128x128.png", "icons/128x128@2x.png", "icons/icon.icns", "icons/icon.ico" ] }
```

描画系

```
`typescript
```

CommandPalette.tsx

```
import React, { useState, useEffect, useRef } from 'react'; import { Command } from '../hooks/useCommandPalette'; import Search from "lucide-react/dist/esm/icons/search";
```

```

interface Props { isOpen: boolean; initialSearch?: string; commands: Command[]; onExecute: (command:
Command) => void; onClose: () => void; theme: 'vs-dark' | 'light'; }

const CommandPalette: React.FC = ({ isOpen, initialSearch = '', commands, onExecute, onClose, theme }) => {
  const [search, setSearch] = useState(''); const [selectedIndex, setSelectedIndex] = useState(0); const inputRef =
  useRef(null); const listRef = useRef(null);

  const filteredCommands = commands.filter(cmd => cmd.name.toLowerCase().includes(search.toLowerCase())
  || (cmd.category && cmd.category.toLowerCase().includes(search.toLowerCase())) );

  useEffect(() => { if (isOpen) { setSearch(initialSearch); setSelectedIndex(0); setTimeout(() =>
  inputRef.current?.focus(), 10); } }, [isOpen, initialSearch]);

  useEffect(() => { setSelectedIndex(0); }, [search]);

  const handleKeyDown = (e: React.KeyboardEvent) => { if (e.key === 'ArrowDown') { e.preventDefault();
  setSelectedIndex(prev => (prev + 1) % filteredCommands.length); } else if (e.key === 'ArrowUp') {
  e.preventDefault(); setSelectedIndex(prev => (prev - 1 + filteredCommands.length) %
  filteredCommands.length); } else if (e.key === 'Enter') { if (filteredCommands[selectedIndex]) {
  onExecute(filteredCommands[selectedIndex]); } } else if (e.key === 'Escape') { onClose(); } };

  useEffect(() => { const selectedElement = listRef.current?.children[selectedIndex] as HTMLElement; if
  (selectedElement) { selectedElement.scrollIntoView({ block: 'nearest' }); } }, [selectedIndex]);

  if (!isOpen) return null;

  const isDark = theme === 'vs-dark';

  return ( <div className="command-palette-overlay" style={{ position: 'fixed', top: 0, left: 0, right: 0, bottom: 0,
  backgroundColor: 'rgba(0,0,0,0.5)', zIndex: 9999, display: 'flex', justifyContent: 'center', padding: '10vh' }}
  onClick={onClose} > <div className="command-palette-container" style={{ width: '600px', maxHeight:
  '400px', backgroundColor: isDark ? '#1e1e1e' : '#ffffff', color: isDark ? '#cccccc' : '#333333', borderRadius: '8px',
  boxShadow: '0 8px 24px rgba(0,0,0,0.3)', display: 'flex', flexDirection: 'column', overflow: 'hidden', border: 1px
  solid ${isDark ? '#333333' : '#dddddd'} }} onClick={e => e.stopPropagation()} > <div style={{ display:
  'flex', alignItems: 'center', padding: '12px 16px', borderBottom: 1px solid ${isDark ? '#333333' :
  '#eeeeee'} }} > <Search size={18} style={{ marginRight: '12px', color: isDark ? '#888' : '#999' }} /> <input ref=
  {inputRef} type="text" placeholder="実行したいコマンドを入力..." value={search} onChange={e =>
  setSearch(e.target.value)} onKeyDown={handleKeyDown} style={{ flexGrow: 1, background: 'none', border:
  'none', color: 'inherit', fontSize: '16px', outline: 'none', fontFamily: 'inherit' }} /> <div ref={listRef} style={{
  overflowY: 'auto', padding: '4px 0' }} > {filteredCommands.length > 0 ? ( filteredCommands.map((cmd, index)
  => ( <div key={cmd.id} onClick={() => onExecute(cmd)} style={{ padding: '8px 16px', cursor: 'pointer', display:
  'flex', justifyContent: 'space-between', alignItems: 'center', backgroundColor: index === selectedIndex ?
  (isDark ? '#0066ff' : '#e6f0ff') : 'transparent', color: index === selectedIndex ? (isDark ? '#ffffff' : '#0066ff') :
  'inherit' }} onMouseEnter={() => setSelectedIndex(index)} > <div style={{ display: 'flex', flexDirection: 'column'
  }} > <span style={{ fontSize: '14px', fontWeight: index === selectedIndex ? 'bold' : 'normal' }}>{cmd.name}
  {cmd.category && <span style={{ fontSize: '11px', opacity: 0.7 }}>{cmd.category}} {cmd.shortcut && ( <span
  style={{ fontSize: '12px', opacity: 0.6, backgroundColor: isDark ? 'rgba(255,255,255,0.1)' : 'rgba(0,0,0,0.05)',
  padding: '2px 6px', borderRadius: '4px' }}> {cmd.shortcut} )) ) ) : ( <div style={{ padding: '16px', textAlign:
  'center', fontSize: '14px', opacity: 0.5 }}> コマンドが見つからないよ... 🤖 )) ); }

```

```
export default CommandPalette;
```

```
,
```

```
`typescript
```

Editor.tsx

```
import React, { useEffect, useRef } from 'react'; import Editor, { OnMount } from '@monaco-editor/react';
```

```
interface MonacoEditorProps { value: string; onChange: (value: string | undefined) => void; fontSize?: number;
fontFamily?: string; theme?: 'vs-dark' | 'light'; language?: string; onMount?: (editor: any) => void; onScroll?:
(top: number, total: number, startLine?: number) => void; snippets?: Record<string, string>; }
```

```
const MonacoEditor: React.FC = ({ value, onChange, fontSize = 16, fontFamily = 'system-ui', theme = 'vs-dark',
language = 'markdown', onMount, onScroll, snippets = {}, }) => { const editorRef = useRef(null); const
monacoRef = useRef(null); const providerRef = useRef(null); const containerRef = useRef(null); const
onScrollRef = useRef(onScroll);
```

```
// onScroll が更新されたら Ref も更新する useEffect(() => { onScrollRef.current = onScroll; }, [onScroll]);
```

```
const handleEditorDidMount: OnMount = (editor, monaco) => { editorRef.current = editor; monacoRef.current
= monaco; editor.onDidChange(e: any) => { // Ref 経由で常に最新の関数を呼ぶ ! if
(onScrollRef.current && e.scrollTopChanged) { const totalHeight = editor.getScrollHeight() -
editor.getLayoutInfo().height; // 可視範囲の先頭行を取得して渡す const ranges = editor.getVisibleRanges();
const startLine = ranges.length > 0 ? ranges[0].startLineNumber : 1; onScrollRef.current(e.scrollTop,
totalHeight, startLine); } }); if (onMount) onMount(editor); }
```

```
// スニペットプロバイダーの設定 useEffect(() => { if (monacoRef.current && language) { if
(providerRef.current) { providerRef.current.dispose(); }
```

```
    providerRef.current =
    monacoRef.current.languages.registerCompletionItemProvider(language, {
      provideCompletionItems: () => {
        const suggestions = Object.entries(snippets).map(([prefix, body]) => {
          let processedBody = body;
          if (processedBody.includes('${currentDate}')) {
            processedBody = processedBody.replace(/\${currentDate}/g, new
            Date().toLocaleDateString());
          }

          return {
            label: prefix,
            kind: monacoRef.current.languages.CompletionItemKind.Snippet,
            insertText: processedBody,
            insertTextRules:
            monacoRef.current.languages.CompletionItemInsertTextRule.InsertAsSnippet,
            documentation: body,
            range: null as any
          };
        });
      }
    });
```

```

    });
    return { suggestions };
  }
});
}

return () => {
  if (providerRef.current) {
    providerRef.current.dispose();
  }
};

```

}, [snippets, language]);

useEffect(() => { if (editorRef.current) { setTimeout(() => editorRef.current.layout(), 100); } }, [fontSize, fontFamily, theme, language]);

return (<div ref={containerRef} style={{ height: '100%', width: '100%', boxSizing: 'border-box', position: 'relative' }} className="monaco-wrapper"> <Editor height="100%" language={language} theme={theme === 'vs-dark' ? 'vs-dark' : 'vs'} defaultValue={value} onChange={onChange} onMount={handleEditorDidMount} options={{ fontSize, fontFamily, minimap: { enabled: false }, automaticLayout: true, // 【復活】 やっぱりこれが一番安定するよ！ lineHeight: 1.7 * fontSize, letterSpacing: 0.04 * fontSize, wordWrap: 'on', links: true, scrollBeyondLastLine: false,

```

    lineNumbers: 'on',
    lineNumbersMinChars: 5,
    lineDecorationsWidth: 10,
    glyphMargin: false,
    folding: false,

    padding: { top: 16, bottom: 16 },

    renderLineHighlight: 'none',
    occurrencesHighlight: 'off',
    selectionHighlight: false,
    renderLineHighlightOnlyWhenFocus: false,
    matchBrackets: 'never',
    hover: { enabled: false },
    lightbulb: { enabled: 'off' as any },
    colorDecorators: false,

    quickSuggestions: {
      other: true,
      comments: false,
      strings: true
    },
    suggestOnTriggerCharacters: true,
    parameterHints: { enabled: false },

    guides: {
      indentation: true
    }

```

```

    },

    scrollbar: {
      vertical: 'visible',
      horizontal: 'hidden',
      useShadows: false,
      verticalScrollbarSize: 10
    },

    acceptSuggestionOnEnter: 'off',
    tabCompletion: 'on',
    // 【重要】fixedOverflowWidgets は IME（日本語入力）を破壊するので OFF に！
    fixedOverflowWidgets: false,
  }}
/>
<style>{`
  /* 検索ウィジェットのボタンや入力を殺さないように CSS を調整！ */
  .monaco-editor-background { outline: none !important; }
  .monaco-editor .inputarea.focus { outline: none !important; }

  .line-numbers {
    font-size: 11px !important;
    opacity: 0.4;
    padding-right: 15px !important;
  }

  /* 検索ウィジェット自体の Z-index を確保 */
  .monaco-editor .find-widget {
    z-index: 2000 !important;
    box-shadow: 0 4px 12px rgba(0,0,0,0.5) !important;
    border: 1px solid var(--border-main) !important;
    pointer-events: all !important;
  }
  .monaco-editor .find-widget .button {
    pointer-events: all !important;
  }
  `}</style>
</div>

```

```
);
```

```
export default MonacoEditor;
```

```
,
```

```
`typescript
```

Preview.tsx

```

import React, { useMemo, useEffect, useRef } from 'react'; import MarkdownIt from 'markdown-it'; import
anchor from 'markdown-it-anchor'; import { openUrl } from '@tauri-apps/plugin-opener'; // 外部リンク用

```

```
import hljs from 'highlight.js'; import DOMPurify from 'dompurify'; // 【追加】サニタイズ用 import 'github-markdown-css/github-markdown.css'; // ダーク/ライト両対応に変更 import 'highlight.js/styles/github-dark.css';
```

```
// markdown-it の初期化 const md = new MarkdownIt({ html: true, linkify: true, typographer: true, breaks: true, highlight: (str, lang) => { if (lang && hljs.getLanguage(lang)) { try { return hljs.highlight(str, { language: lang }).value; } catch (__) {} } return ''; // デフォルトのエスケープを使用 } });
```

```
// 行番号注入プラグイン function injectLineNumbers(md: MarkdownIt) { md.core.ruler.push('inject_line_numbers', (state) => { state.tokens.forEach((token) => { if (token.map && (token.type.endsWith('_open') || token.type === 'fence' || token.type === 'code_block')) { const line = token.map[0] + 1; token.attrSet('data-line', String(line)); token.attrJoin('class', 'source-line'); } } }); } md.use(injectLineNumbers);
```

```
// anchor プラグインの使用 md.use(anchor, { permalink: false, slugify: (s: string) => encodeURIComponent(String(s).trim().toLowerCase().replace(/s+/g, '-')) });
```

```
// コードブロックのカスタムレンダラー（コピーボタン付き） md.renderer.rules.fence = (tokens, idx, options, _env, _self) => { const token = tokens[idx]; const code = token.content; const lang = token.info.trim(); const language = lang || 'text';
```

```
// ハイライト処理 let highlighted = ''; if (options.highlight) { highlighted = options.highlight(code, lang, ''); }
```

```
if (!highlighted) { highlighted = md.utils.escapeHtml(code); }
```

```
return <div class="code-block-wrapper" data-lang="${language.toUpperCase()}"> <button class="copy-btn" data-code="${encodeURIComponent(code.trim())}">Copy</button> <pre><code class="hljs language-${language}">${highlighted}</code></pre> </div> ;;
```

```
interface PreviewProps { content: string; fontSize?: number; fontFamily?: string; theme?: 'vs-dark' | 'light'; openExternalBrowser?: boolean; onScroll?: (top: number, total: number) => void; }
```

```
const Preview: React.FC = ({ content, fontSize = 16, fontFamily = 'system-ui', theme = 'vs-dark', openExternalBrowser = true, onScroll }) => { const containerRef = useRef(null); const isAutoScrolling = useRef(false);
```

```
// スクロールイベントの監視 useEffect(() => { const el = containerRef.current; if (!el || !onScroll) return;
```

```
const handleScrollEvent = () => {
  // 内部リンクによる自動スクロール中は同期をスキップ！
  if (isAutoScrolling.current) return;

  const top = el.scrollTop;
  const total = el.scrollHeight - el.clientHeight;
  onScroll(top, total);
};

el.addEventListener('scroll', handleScrollEvent);
return () => el.removeEventListener('scroll', handleScrollEvent);
```

```
}, [onScroll]);
```

```
// 独自アンカーライン (text) を に変換 const preprocessedContent = useMemo(() => { return
content.replace(/(.+?)/g, '$1'); }, [content]);
```

```
const html = useMemo(() => { try { const rawHtml = md.render(preprocessedContent); // XSS対策 :
DOMPurify でサニタイズ。独自属性は許可する。 return DOMPurify.sanitize(rawHtml, { ADD_ATTR: ['data-
line', 'data-code', 'data-lang'] }); } catch (e) { return Parse Error: ${e}; }, [preprocessedContent]);
```

```
useEffect(() => { const handleEvents = (e: MouseEvent) => { const target = e.target as HTMLElement;
```

```
    // 1. コピーボタンの処理
    if (target.classList.contains('copy-btn')) {
        const code = decodeURIComponent(target.getAttribute('data-code') || '');
        navigator.clipboard.writeText(code).then(() => {
            const originalText = target.innerText;
            target.innerText = 'Copied!';
            target.classList.add('success');
            setTimeout(() => {
                target.innerText = originalText;
                target.classList.remove('success');
            }, 2000);
        });
        return;
    }

    // 2. リンククリックの処理
    const anchorEl = target.closest('a');
    if (anchorEl) {
        const href = anchorEl.getAttribute('href');
        if (!href) return;

        if (href.startsWith('http')) {
            // 外部リンク
            if (openExternalBrowser) {
                e.preventDefault();
                openUrl(href).catch(() => {
                    window.open(href, '_blank');
                });
            }
        } else if (href.startsWith('#')) {
            // 内部アンカーリンク
            e.preventDefault();
            const rawId = href.slice(1);
            const decodedId = decodeURIComponent(rawId);

            try {
                // IDまたはname属性で要素を検索
                // 1. 生のID (エンコードされたまま)
                // 2. デコードされたID
                // 3. name属性 (エンコード)
                const targetEl = document.getElementById(rawId) ||
```

```

        document.getElementById(decodedId)_)||.
containerRef.current?.querySelector(`[id="${CSS.escape(rawId)}"]`)||.
containerRef.current?.querySelector(`[id="${CSS.escape(decodedId)}"]`)||.
containerRef.current?.querySelector(`a[name="${CSS.escape(rawId)}"]`);

    if (targetEl) {
        isAutoScrolling.current = true;
        targetEl.scrollIntoView({ behavior: 'smooth', block: 'start' });
        // アニメーションが終わる頃にロックを解除
        setTimeout(() => { isAutoScrolling.current = false; }, 800);
    }
} catch (err) {
    console.error("Internal link navigation failed:", err);
}
}.
}.
}.
};

const el = containerRef.current;
if (el) {
    el.addEventListener('click', handleEvents, true);
    return () => el.removeEventListener('click', handleEvents, true);
}

```

```
}, [openExternalBrowser, html]);
```

```
return ( <div className="preview-container" data-theme={theme} style={{ height: '100%', width: '100%',
overflow: 'hidden', backgroundColor: theme === 'vs-dark' ? '#1e1e1e' : '#faf9f6'}} >);
```

```
export default TabBar;
```

```
`
```

```
`css
```

App.css

```
:root { --primary: #0066ff; --primary-hover: #0052cc; --transition-speed: 0.15s; }
```

```
/* ダークモード */ [data-theme="vs-dark"] { --bg-main: #1e1e1e; --bg-sidebar: #252526; --bg-actionbar:
#2d2d2d; --bg-tab-active: #1e1e1e; --bg-tab-inactive: #2d2d2d; --bg-hover: #2a2d2e; --bg-input: #3c3c3c;
--text-main: #d4d4d4; --text-muted: #858585; --text-tab: #969696; --text-tab-active: #ffffff;
```

```
--border-main: #333333; --border-focus: #0066ff; }
```

```
/* 目に優しいライトモード (文字をくっきりさせたよ!) / [data-theme="light"] { --bg-main: #faf9f6; / 背景は
そのまま */ --bg-sidebar: #f2f1ee; --bg-actionbar: #e8e7e3; --bg-tab-active: #ffffff; --bg-tab-inactive:
```

```
#e8e8e8; --bg-hover: #e0deda; --bg-input: #ffffff;
```

```
--text-main: #1a1a1c; /* 【修正】文字を濃くして読みやすくしたよ！ */ --text-muted: #5a5a5c; --text-tab: #616161; --text-tab-active: #1a1a1c;
```

```
--border-main: #d1d1d1; --border-focus: #0066ff; }
```

```
body { margin: 0; padding: 0; overflow: hidden; height: 100vh; width: 100vw; background-color: var(--bg-main); color: var(--text-main); font-family: -apple-system, "BlinkMacSystemFont", "Hiragino Sans", "Hiragino Kaku Gothic ProN", "Yu Gothic", "Meiryo", sans-serif; line-height: 1.7; letter-spacing: 0.04em; transition: background-color var(--transition-speed), color var(--transition-speed); }
```

```
#root { height: 100vh; } .container { height: 100%; width: 100%; display: flex; overflow: hidden; }
```

```
[data-theme="vs-dark"] ::-webkit-scrollbar { width: 10px; height: 10px; } [data-theme="vs-dark"] ::-webkit-scrollbar-track { background: var(--bg-main); } [data-theme="vs-dark"] ::-webkit-scrollbar-thumb { background: #333; } [data-theme="vs-dark"] ::-webkit-scrollbar-thumb:hover { background: #444; }
```

```
[data-theme="light"] ::-webkit-scrollbar { width: 10px; height: 10px; } [data-theme="light"] ::-webkit-scrollbar-track { background: var(--bg-main); } [data-theme="light"] ::-webkit-scrollbar-thumb { background: #ccc; } [data-theme="light"] ::-webkit-scrollbar-thumb:hover { background: #bbb; }
```

```
`
```

```
`html
```

index.html

```
`
```

その他

```
`markdown
```

LICENSE.md

LICENSE

Project License

MIT License

Copyright (c) 2026 YuzukoUnderson

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Third-Party Licenses

This software uses the following open-source libraries:

Frontend Dependencies (npm)

- @monaco-editor/react: MIT License
- @tauri-apps/api: MIT or Apache-2.0 License
- @tauri-apps/plugin-dialog: MIT or Apache-2.0 License
- @tauri-apps/plugin-opener: MIT or Apache-2.0 License
- @tauri-apps/plugin-shell: MIT or Apache-2.0 License
- github-markdown-css: MIT License
- highlight.js: BSD-3-Clause License
- lucide-react: ISC License
- markdown-it: MIT License
- markdown-it-anchor: MIT License
- react: MIT License
- react-dom: MIT License

Backend Dependencies (Cargo)

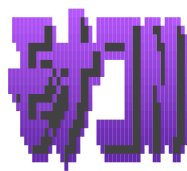
- tauri: MIT or Apache-2.0 License
- tauri-plugin-opener: MIT or Apache-2.0 License
- serde: MIT or Apache-2.0 License
- serde json: MIT or Apache-2.0 License
- encoding_rs: MIT or Apache-2.0 License
- tauri-plugin-dialog: MIT or Apache-2.0 License
- tauri-plugin-shell: MIT or Apache-2.0 License

、
-

`markdown`

README.md

モナコノート / Monaco Note



[日本語](#) | [English](#)

日本語

概要

モナコノートは、VSCode のコアエディタである Monaco Editor を内包したシンプルなデスクトップ用メモ帳です。Tauri と React をベースに構築されており、ネイティブアプリの軽快な動作と、Monaco Editor による快適なテキスト編集体験を提供します。

主な機能

- テキスト編集: Monaco Editor による高度な編集機能（マルチカーソル、検索・置換等）。
- 多文字コード対応: UTF-8 に加え、Shift-JIS や EUC-JP の読み書きをサポート。
- 自動保存・復元: タイピングを止めると自動保存。未保存のタブやアプリの状態を次回起動時に完全復元。
- コマンドパレット: Ctrl+Shift+P ですべての機能にアクセス可能。
- カスタマイズ: エディタとプレビューそれぞれのフォント設定やカスタムスニペットを適用可能。
- Markdown プレビュー（付随機能）: Markdown 形式のテキストをリアルタイムでプレビューする機能も備えています。

技術スタック

- Frontend: React 19, TypeScript, Vite
- Backend: Tauri v2, Rust
- Editor: Monaco Editor (@monaco-editor/react).
- Styling: GitHub Markdown CSS, Lucide React, CSS Modules

開発者向けセットアップ

ビルドには以下の環境が必要です：

- Node.js (v18以上推奨)
- Rust (最新の stable バージョン)
- WebView2 (Windows の場合)

```
# 依存関係のインストール  
npm install
```

```
# 開発モードの起動（デバッグ用）  
npm run tauri dev
```

```
# 本番用パッケージのビルド（exe/msi 等の生成）  
npm run tauri build
```

English

Overview

Monaco Note is a simple desktop notepad that integrates Monaco Editor, the core of VSCode. Built with Tauri and React, it combines the snappiness of a native application with the smooth text editing experience of Monaco Editor.

Key Features

- Text Editing: Advanced editing capabilities (multi-cursor, find/replace, etc.) powered by Monaco Editor.
- Multi-Encoding Support: Seamlessly read and write various encodings including UTF-8, Shift-JIS, and EUC-JP.
- State Preservation: Automatically saves work and restores unsaved tabs, scroll positions, and layout across sessions.
- Command Palette: Quick access to all features via Ctrl+Shift+P.
- Extensive Customization: Independent font settings and custom snippets.
- Markdown Preview (Extra): Includes an optional real-time Markdown previewing capability.

Tech Stack

- Frontend: React 19, TypeScript, Vite
- Backend: Tauri v2, Rust
- Editor: Monaco Editor (@monaco-editor/react).
- Styling: GitHub Markdown CSS, Lucide React, CSS Modules

Setup for Developers

The following environments are required for building:

- Node.js (v18 or higher recommended).
- Rust (latest stable version).
- WebView2 (For Windows).

```
# Install dependencies  
npm install
```

```
# Run in development mode (for debugging).  
npm run tauri dev
```

```
# Build the production package (generates exe/msi, etc.).  
npm run tauri build
```

License

This project is licensed under the MIT License - see the LICENSE.md file for details.

```
# 依存関係のインストール  
npm install  
---  
```bash  
開発モードの起動 (デバッグ用)
npm run tauri dev

```bash  
# 本番用パッケージのビルド (exe/msi 等の生成)  
npm run tauri build  
---  
---  
  
<a name="english"></a>  
## English  
  
### Overview  
Monaco Note is a simple desktop notepad that integrates Monaco Editor, the core of  
VSCode.  
Built with Tauri and React, it combines the snappiness of a native application  
with the smooth text editing experience of Monaco Editor.  
  
### Key Features  
- Text Editing: Advanced editing capabilities (multi-cursor, find/replace, etc.)  
powered by Monaco Editor.  
- Multi-Encoding Support: Seamlessly read and write various encodings including  
UTF-8, Shift-JIS, and EUC-JP.  
- State Preservation: Automatically saves work and restores unsaved tabs, scroll  
positions, and layout across sessions.  
- Command Palette: Quick access to all features via Ctrl+Shift+P.  
- Extensive Customization: Independent font settings and custom snippets.  
- Markdown Preview (Extra): Includes an optional real-time Markdown previewing  
capability.  
  
### Tech Stack  
- Frontend: React 19, TypeScript, Vite
```

- Backend: Tauri v2, Rust
- Editor: Monaco Editor ([@monaco-editor/react](#))
- Styling: GitHub Markdown CSS, Lucide React, CSS Modules

Setup for Developers

The following environments are required for building:

- Node.js (v18 or higher recommended)
- Rust (latest stable version)
- WebView2 (For Windows)

```
```bash
```

```
Install dependencies
```

```
npm install
```

```
```
```

```
```bash
```

```
Run in development mode (for debugging).
```

```
npm run tauri dev
```

```
```
```

```
```bash
```

```
Build the production package (generates exe/msi, etc.).
```

```
npm run tauri build
```

```
```
```

```
---
```

License

This project is licensed under the MIT License - see the LICENSE.md file [for details](#).

```
`
```

```
-
```